

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2 Programación Dinámica

12 de octubre de 2025

Agustin Ferronato
111991

Federico Parsons
111250

Axel Alvarado
107679

Ignacio Sebastián Oviedo
109821

1. Introducción

En el presente informe se realiza el análisis de un algoritmo de programación dinámica, diseñado para encontrar la solución óptima al problema que se describe a continuación.

Problema: Tenemos un conjunto de minutos $1, 2, \dots, n$. En cada minuto i , llega una cantidad x_i de enemigos. Se define una función monótona creciente $f(\cdot)$, tal que $f(j)$ representa el número máximo de enemigos que pueden ser eliminados al efectuar una defensa luego de esperar j minutos desde el último ataque. Si se ataca en el minuto i después de haber esperado j minutos, la cantidad de enemigos eliminados será:

$$\min(x_i, f(j))$$

El objetivo es determinar en qué minutos realizar los ataques para **maximizar el número total de enemigos eliminados** a lo largo de los n minutos.

2. Algoritmo

2.1. Planteos

Se modeló el problema como una **barrera permeable** ya que los enemigos que llegan mientras la barrera está en recarga no se acumulan en ella; es decir, esos enemigos avanzan hacia Ba Sing Se y no afectan directamente el cálculo del ataque actual. El objetivo de la barrera es **maximizar la cantidad de enemigos eliminados**, lo que equivale a **minimizar los enemigos que logran llegar a la ciudad**. Cuando empezamos a pensar con el algoritmo, observamos una similitud con el *problema del cambio de monedas*, ya que para cada minuto i se busca determinar el óptimo recorriendo una lista creciente de posibles tiempos de espera, y combinando ese valor con un óptimo previamente calculado para subproblemas anteriores, de manera muy similar a cómo se calcula cada uno de los cambios óptimos recorriendo las denominaciones de un sistema monetario.

2.2. Algoritmo planteado

Se recorre la lista de ataques, y por cada i se calcula el valor óptimo correspondiente. El óptimo de la posición i se define como el máximo entre todas las posibles esperas t , de la suma entre el valor óptimo previo $OPT[i - t]$ y el mínimo entre $f(t)$ y x_i , que representa la cantidad de enemigos eliminados al atacar en ese momento. La ecuación de recurrencia es la siguiente:

$$OPT[i] = \begin{cases} 0 & \text{si } i = 0, \\ \max_{t \in [1, i]} (OPT[i - t] + \min(f(t), x_i)) & \text{si } i > 0 \end{cases}$$

El algoritmo utiliza **programación dinámica**, ya que emplea **memoización** en la lista de valores óptimos OPT . En esta implementación se adopta un enfoque **bottom-up**, calculando las soluciones de los subproblemas base hacia los más grandes.

El código luce de la siguiente manera:

```
1 def algorithm(xi: list[int], f: list[int]) -> tuple[int, list[str]]:
2
3     n: int = len(xi)
4     OPT: list[int] = [0] * (n + 1)
5
6     for i in range(1, n + 1):
7         max_value: int = -1
8         for j in range(1, i + 1):
9             current_value = OPT[i - j] + min(f[j - 1], xi[i - 1])
10            if max_value < current_value:
11                max_value = current_value
12            OPT[i] = max_value
13
14     return OPT[n], reconstruction(xi, f, OPT)
```

A continuación, se muestra el código encargado de reconstruir la solución del problema, es decir, determinar la secuencia de cargas/ataques que permite alcanzar la cantidad máxima de enemigos eliminados:

```
1 def reconstruction(xi: list[int], f: list[int], OPT: list[int]) -> list[str]:
2     index: int = len(xi)
3     solution: list[str] = []
4
5     while index > 0:
6         for j in range(1, index + 1):
7             if OPT[index] == OPT[index - j] + min(f[j - 1], xi[index - 1]):
8                 solution.append("Atacar")
9                 solution.extend(["Cargar"] * (j - 1))
10                index -= j
11                break
12
13     solution.reverse()
14     return solution
```

3. Optimalidad, ejecuciones y comparación con algoritmo de fuerza bruta

3.1. Algoritmo de fuerza bruta

El trabajo cuenta con un script encargado no tan solo de comparar el resultado del algoritmo con los otorgados por la cátedra, sino también de generar sets de datos junto a su resultado óptimo. Para lograr que esto último se cumpla sin incurrir en la utilización del algoritmo propuesto (lo cual desencadenaría en una lógica circular), codificamos un algoritmo de fuerza bruta que, en todo momento dado, nos asegura obtener la solución óptima a nuestro problema.

El algoritmo es el siguiente:

```
1 def algorithm_by_bf(xi, f, index, last_attack, current_solution, best_solution):
2
3     if index == len(xi):
4         if sum(current_solution) > sum(best_solution):
5             return current_solution[:]
6         return best_solution
7
8     current_xi = xi[index]
9     current_f = f[index - last_attack]
10
11     current_solution.append(min(current_xi, current_f))
12     best_solution = algorithm_by_bf(xi, f, index + 1, index + 1, current_solution,
13                                     best_solution)
14
15     current_solution.pop()
16
17     return algorithm_by_bf(xi, f, index + 1, last_attack, current_solution,
18                             best_solution)
```

Principalmente, se utilizó para comparar sets de datos de tamaño chico ($n \leq 10$), ya que el tiempo de ejecución del algoritmo se vuelve muy grande a medida que aumenta el tamaño de n (recordar que los algoritmos de fuerza bruta poseen una complejidad de $O(2^n)$).

3.2. Optimalidad según la variabilidad de los valores de x_i y $f(j)$

Si bien puede resultar obvio, vale aclarar que la variabilidad de los valores de x_i y $f(j)$ **no afectan a la optimalidad del algoritmo**.

En primer lugar, no estamos analizando una estrategia greedy la cual podría ser refutada con contraejemplos. En cambio, la técnica de diseño utilizada se basa en aprovechar soluciones previamente calculadas a problemas más chicos de la misma naturaleza. Esto permite, en cada paso, seleccionar la mejor decisión entre todas las opciones disponibles (atacar ahora y no haber esperado respecto al ataque anterior, esperar y haber atacado dos minutos atrás, etcétera).

Incluso si decidieramos modificar las condiciones o el dominio de nuestro problema, el algoritmo seguiría siendo óptimo.

Por ejemplo, se analizaron diversas ejecuciones con sets de datos donde la función f era constante. Por supuesto, el algoritmo llega a la solución óptima más intuitiva: atacar siempre. Otra opción que genera el mismo resultado es la de hacer que f sea una función monótona decreciente (por más de que pierda sentido con respecto al enunciado: si cargo energía debería poder eliminar más enemigos). De la misma manera, si f simplemente es un arreglo sin un orden estricto (al igual que los valores x_i), llegamos también a la solución óptima del problema.

Es decir, cualquier posible combinación u orden de los valores de x_i y/o f **no afecta a la optimalidad del algoritmo**, por más que su modificación contradiga al dominio y contexto del problema. Naturalmente, no fueron considerados valores que carecen de un sentido "físico", como llegadas de enemigos no positivas o no enteras, o ataques que eliminen una cantidad negativa de enemigos.

4. Complejidad computacional y mediciones

4.1. Justificación de la complejidad

La complejidad temporal total del algoritmo, incluyendo tanto el cálculo de la solución óptima como su reconstrucción, es:

$$T(n) = O(n^2)$$

Justificación: El algoritmo principal recorre la lista de valores x_i en n iteraciones externas. En cada una de ellas evalúa todas las posibles esperas t entre 1 e i , lo que implica un recorrido interno de orden $O(n)$. Por lo tanto, la complejidad temporal del cálculo es:

$$T(n) = O(n) \times O(n) = O(n^2)$$

El algoritmo de reconstrucción presenta una lógica análoga a la del cálculo de la solución: inicia desde el último elemento del arreglo OPT y, para cada posición donde se ejecuta un ataque, identifica el índice anterior desde el cual proviene el valor óptimo. Como determinar dicha posición demanda un tiempo $O(n)$ y este proceso puede repetirse hasta n veces, la complejidad temporal de la reconstrucción también es $O(n^2)$.

En consecuencia, la complejidad temporal general se mantiene en:

$$T(n) = O(n^2)$$

Por otro lado, la complejidad espacial es:

$$S(n) = O(n)$$

ya que se utiliza un único arreglo OPT de tamaño lineal para almacenar los valores óptimos de cada subproblema.

4.2. Mediciones

Con el objetivo de verificar empíricamente la complejidad computacional, se realizaron diversas mediciones utilizando la técnica de cuadrados mínimos. Para cada ejecución, se tomaron un total de 25 muestras aleatorias, para tamaños de entrada desde $n = 1000$ hasta $n = 10000$. Además, para cada medición, se realizaron un total de 20 ejecuciones, con el fin de reducir el ruido del resultado final (el promedio de dichas ejecuciones).

Los valores de dichas muestras son dos listas de la forma $x[i] = x_i$ y $F[i] = f(i)$.

Como primera medicion, generamos, para cada muestra, un set de datos aleatorio donde los valores de x_i varían entre 100 y 1000, y los de $f(i)$, entre 50 y 2000. Estos últimos fueron ordenados crecientemente, tal como lo indica el enunciado.

Al ajustar dicha medicion y compararla con la función $T(n) = n^2$ (usando la técnica de cuadrados mínimos), obtuvimos un error cuadrático total $r \approx 0,12028$. A su vez, realizamos un ajuste con las funciones $T(n) = n^3$ y $T(n) = n \log n$, siendo el error cuadrático total $r \approx 0,92829$ y $r \approx 2,99997$, respectivamente.

Los gráficos de ajuste para las tres funciones se observan en las figuras 1, 2 y 3. Además, en la figura 4, se muestra la comparación del error obtenido para cada valor de entrada en las tres funciones.

Se observa a simple vista como la función $T(n) = n^2$ es la que mejor se ajusta a los tiempos de ejecución del algoritmo dados los datos de entrada mencionados anteriormente.

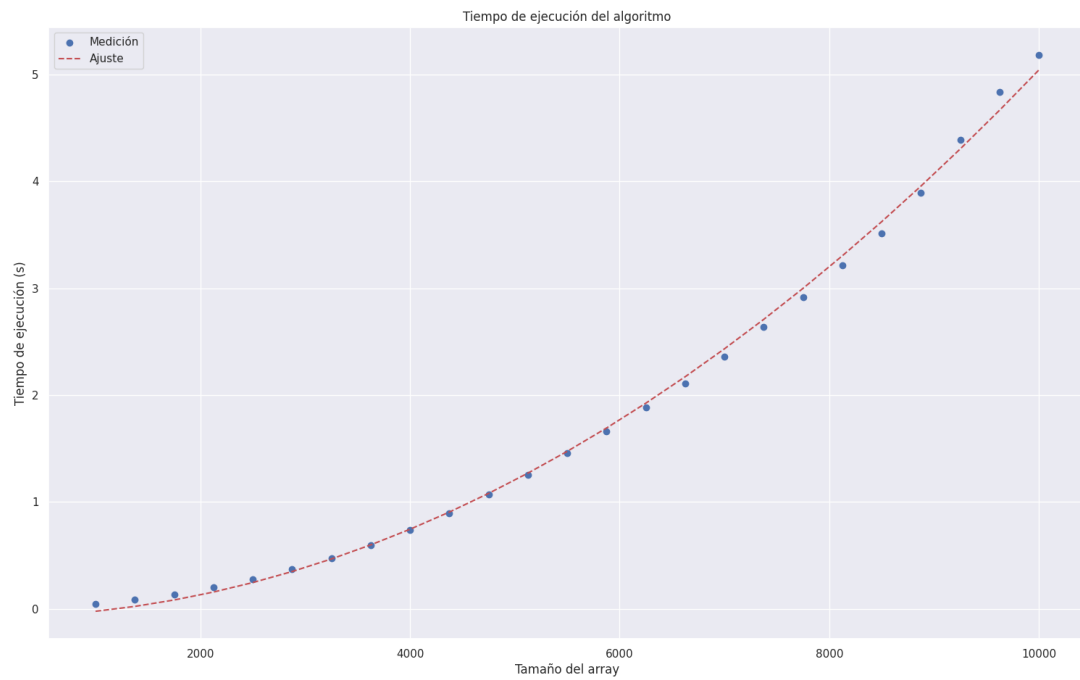


Figura 1: Ajuste del tiempo de ejecución para $T(n) = n^2$

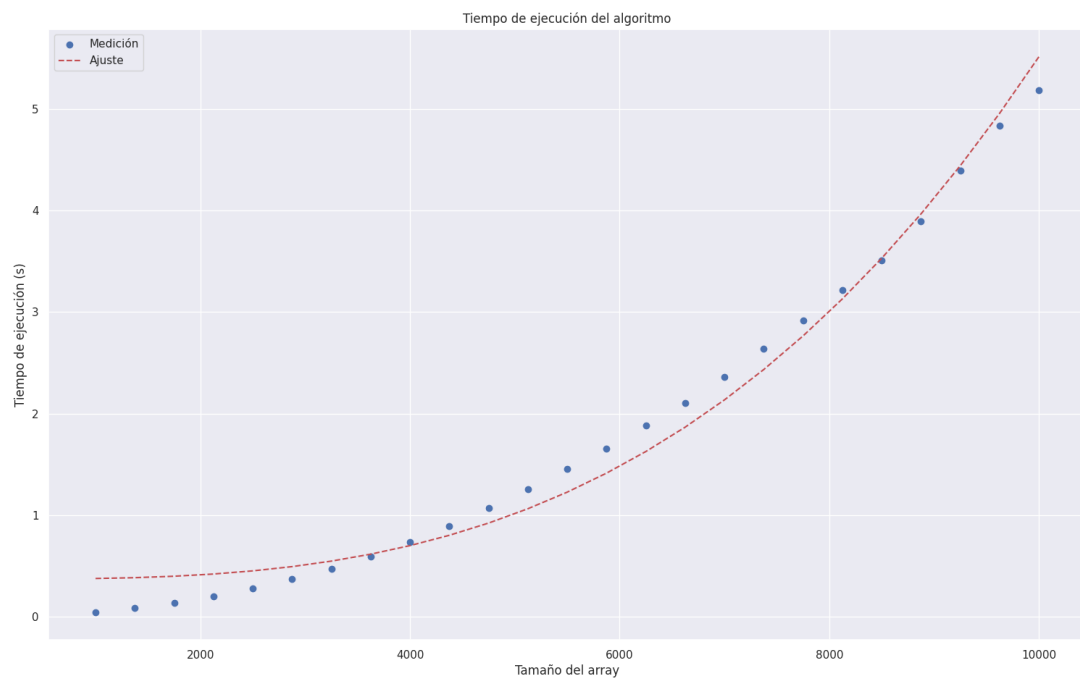


Figura 2: Ajuste del tiempo de ejecución para $T(n) = n^3$.

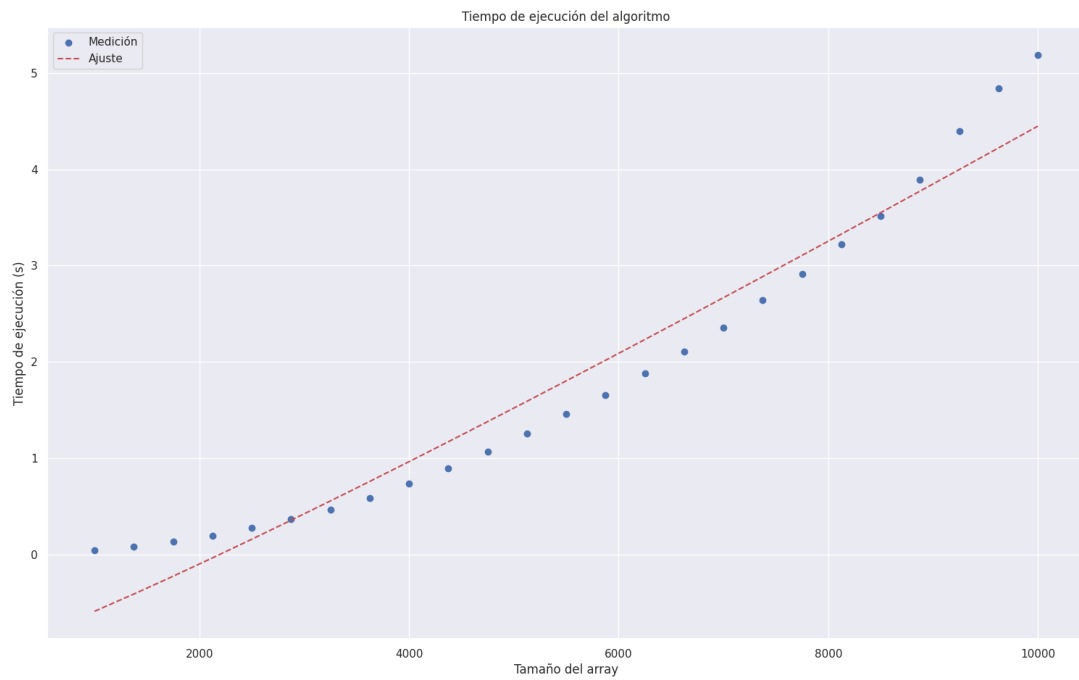


Figura 3: Ajuste del tiempo de ejecución para $T(n) = n \log n$.

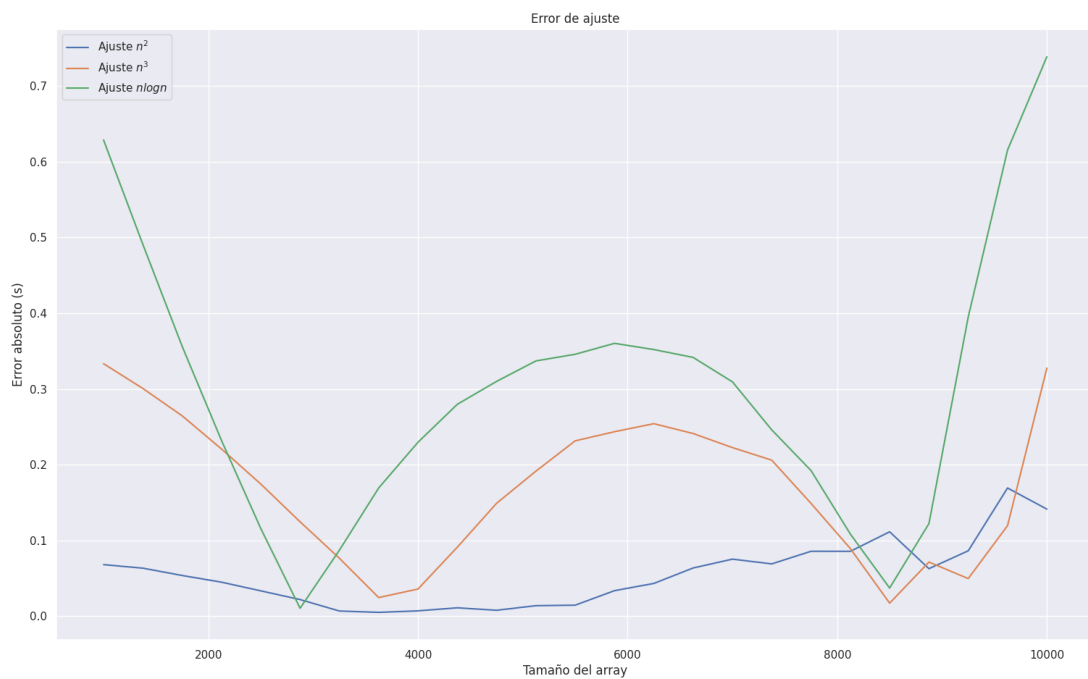


Figura 4: Comparación de error de ajuste para las tres funciones.

4.3. Análisis de la complejidad en función a la variabilidad de los valores de las llegadas de enemigos y recargas.

En comparación al trabajo anterior, no hay optimizaciones realizadas (a priori) que permitan mejorar los tiempos del algoritmo en función de que valores tomen los $f(j)$ y x_i . Esto ocurre ya que, en cualquier escenario, la lista recorre todas las opciones anteriores para seleccionar el óptimo de cada minuto. Por ende, la complejidad seguirá siendo exactamente la misma en cada circunstancia.

De todas formas, se realizaron algunas mediciones para verificar empíricamente que la complejidad se mantiene en $T(n) = O(n^2)$ en diversos escenarios. Para cada uno, se especifica el error cuadrático obtenido al ajustar los tiempos de ejecución con la complejidad indicada (los tamaños de entrada y condiciones de ejecución son las mismas que las mencionadas en la sección anterior).

- f es una función constante: $r \approx 0,08862$
- Los valores x_i están ordenados crecientemente: $r \approx 0,02726$
- Tanto f como x_i permanecen constantes: $r \approx 0,052722$
- Los valores x_i están ordenados de forma decreciente: $r \approx 0,009142$

5. Conclusiones

En conclusión, se demostró que el algoritmo de programación dinámica planteado para resolver el problema obtiene siempre una solución.

Además, se realizaron varias mediciones utilizando la técnica de cuadrados mínimos para verificar empíricamente que nuestro algoritmo tiene una complejidad de $O(n^2)$, y que la variabilidad de los valores de x_i y de $f(\cdot)$ no afecta de manera significativa el tiempo de ejecución final del algoritmo.

Asimismo, se justificó la razón por la cual nuestro algoritmo corresponde a programación dinámica(utilizando memoización y una estrategia bottom-up). Con esto nos referimos a que es conveniente almacenar los resultados parciales (subproblemas) y construir la solución acumulando los valores máximos posibles, de manera que se optimiza la selección de los elementos de acuerdo a sus tiempos y beneficios, priorizando aquellos con mayor contribución acumulada.

5.1. Planteos y observaciones

Antes de implementar optimizaciones concretas, conviene analizar algunas propiedades de la función $f(\cdot)$ y de los valores de enemigos x_i :

- Calcular el valor mínimo de $f(\cdot)$ es trivial, ya que la función es monótonamente creciente:

$$\min_{i \in [0, n]} f(i) = f(0)$$

- También puede determinarse el máximo número de enemigos que llega en algún minuto:

$$x_{\max} = \max_{i \in [1, n]} x_i$$

- Si se cumple que $f(0) \geq x_{\max}$, entonces para toda posición i se verifica:

$$\min_{t \in [0, i]} (x_i, f(t)) = x_i$$

En ese caso, la solución óptima es trivial: conviene atacar en cada instante sin esperar, ya que cada ataque elimina a todos los enemigos que lleguen.

5.2. Versión mejorada: corte de iteración

Partiendo de la observación anterior, se puede optimizar el cálculo del óptimo en el caso general. Al recorrer los posibles tiempos de espera t para calcular $OPT[i]$, una vez que $f(t) \geq x_i$, no es necesario seguir iterando, ya que los ataques posteriores no aportarán un valor mayor. De este modo, se define:

$$OPT[i] = \begin{cases} 0, & \text{si } i = 0, \\ \max_{t \in [1, t^*]} (OPT[i - t] + f(t)), & \text{si } i \geq 1 \text{ y } f(t) < x_i, \\ x_i + OPT[OPT_{\max}[i - t^*]], & \text{si } i \geq 1 \text{ y } f(t^*) \geq x_i, \end{cases}$$

donde t^* es el primer t tal que $f(t) \geq x_i$, y se define una lista auxiliar de índices:

$$OPT_{\max}[k] = \arg \max_{j \in [0, k]} OPT[j].$$

Esta optimización permite cortar la iteración en cuanto se supera el umbral útil $f(t^*)$, reduciendo así cálculos innecesarios sin alterar la corrección del algoritmo.

```
1 def algorithm(xi: list[int], f: list[int]) -> tuple[int, list[str]]:
2
3     n: int = len(xi)
4     OPT: list[int] = [0] * (n + 1)
5     OPT_max: list[int] = [0] * (n + 1)
6     for i in range(1, n + 1):
7         max_value: int = -1
8         t_de_corte: int = -1
9         for j in range(1, i + 1):
10             current_value = OPT[i - j] + min(f[j - 1], xi[i - 1])
11             if max_value < current_value:
12                 max_value = current_value
13                 t_de_corte = j
14             if f[j - 1] >= xi[i - 1]:
15                 t_de_corte = j
16                 current_value = OPT[OPT_max[i - j]] + xi[i - 1]
17                 if max_value < current_value:
18                     max_value = current_value
19                 break
20
21         OPT[i] = max_value
22         OPT_max[i] = i if OPT[i] >= OPT[OPT_max[t_de_corte]] else OPT_max[
23             t_de_corte]
24
25     return OPT[n], reconstruction(xi, f, OPT)
```

5.2.1. Peor caso

En el peor caso, cuando $f(t) < x_i$ para todos los valores de t , el algoritmo debe recorrer todos los posibles tiempos de espera. En este escenario, la complejidad temporal es:

$$T(n) = O(n^2)$$

5.2.2. Mejor caso

En el mejor caso, cuando $f(0) \geq x_i$, la condición de corte se cumple de forma inmediata y cada iteración se resuelve en tiempo constante, alcanzando así una complejidad asintótica mínima de:

$$T(n) = \Omega(n)$$

5.2.3. Caso promedio y crecimiento relativo

Sea $g: \mathbb{N} \rightarrow \mathbb{N}$ la función que genera la cantidad de enemigos en cada minuto antes de mezclarlos o permutarlos, de modo que

$$x_i = \text{alguna permutación de } g(i), \quad i = 1, 2, \dots, n.$$

De esta forma, aunque los x_i puedan no ser monótonos después de la mezcla, su origen está definido por la función $g(\cdot)$, que crece de manera monótona.

Para analizar el crecimiento relativo de la función de defensa $f(\cdot)$ frente a la generación de enemigos, se considera el límite

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0, & \text{si } g(n) \text{ crece mucho más rápido que } f(n), \\ \infty, & \text{si } f(n) \text{ crece mucho más rápido que } g(n), \\ 1, & \text{si crecen al mismo ritmo asintótico.} \end{cases}$$

Observación: al estudiar el límite cuando $n \rightarrow \infty$, los factores constantes no afectan el resultado, ya que

$$\lim_{n \rightarrow \infty} \frac{c \cdot f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}, \quad c > 0.$$

De esta manera, se puede formalizar el caso promedio como

$$T(n) = \begin{cases} \Theta(n), & \text{si } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty, \\ \Theta(n^2), & \text{caso contrario.} \end{cases}$$

5.3. Espacio

Por último, respecto al espacio utilizado, se emplean únicamente dos arreglos auxiliares de tamaño $n + 1$, por lo que la complejidad espacial se mantiene lineal:

$$S(n) = O(n)$$

5.4. Mediciones sobre la versión mejorada

En comparación con lo analizado en la sección 4.3, en este caso la variabilidad de los valores de f y x_i **puede mejorar sustancialmente los tiempos de ejecución del algoritmo.**

5.4.1. Caso lineal

De acuerdo con lo mencionado anteriormente, si $f(\cdot)$ acota a $g(\cdot)$, la complejidad del algoritmo es lineal. Para comprobar esto, se realizaron diversas mediciones sobre conjuntos de datos que cumplieran con dicha condición. Como se observa en las figuras 5 y 6, la función $T(n) = n$ es la que mejor se ajusta a los tiempos de ejecución del algoritmo propuesto.

En este caso favorable, se obtuvo un error cuadrático medio de $r \approx 1,06 \times 10^{-6}$, en comparación con el ajuste mediante $T(n) = n^2$, cuyo error fue $r \approx 5,7 \times 10^{-6}$.

El error obtenido para cada uno de los valores de entrada, en ambas funciones, puede observarse en la figura 7.

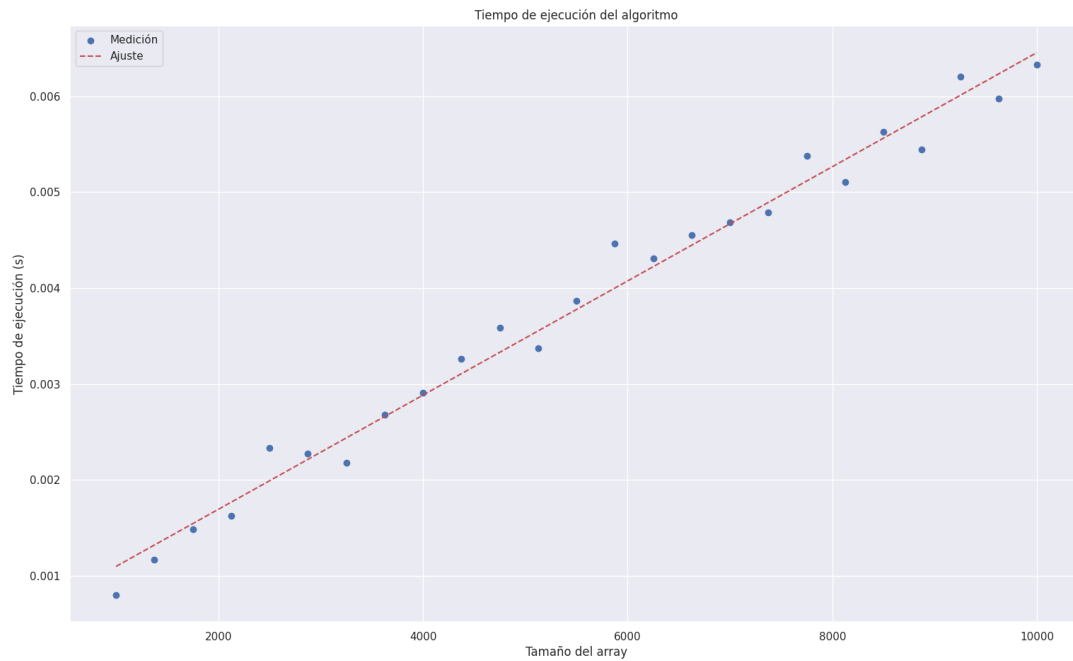


Figura 5: Ajuste del tiempo de ejecución para $T(n) = n$.

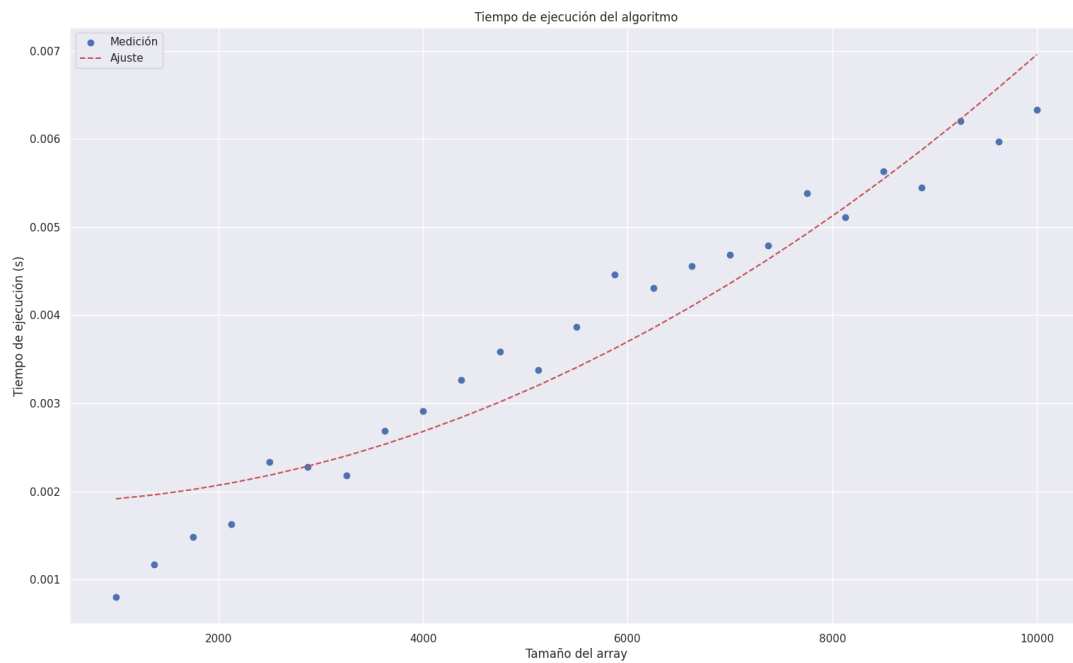


Figura 6: Ajuste del tiempo de ejecución para $T(n) = n^2$.

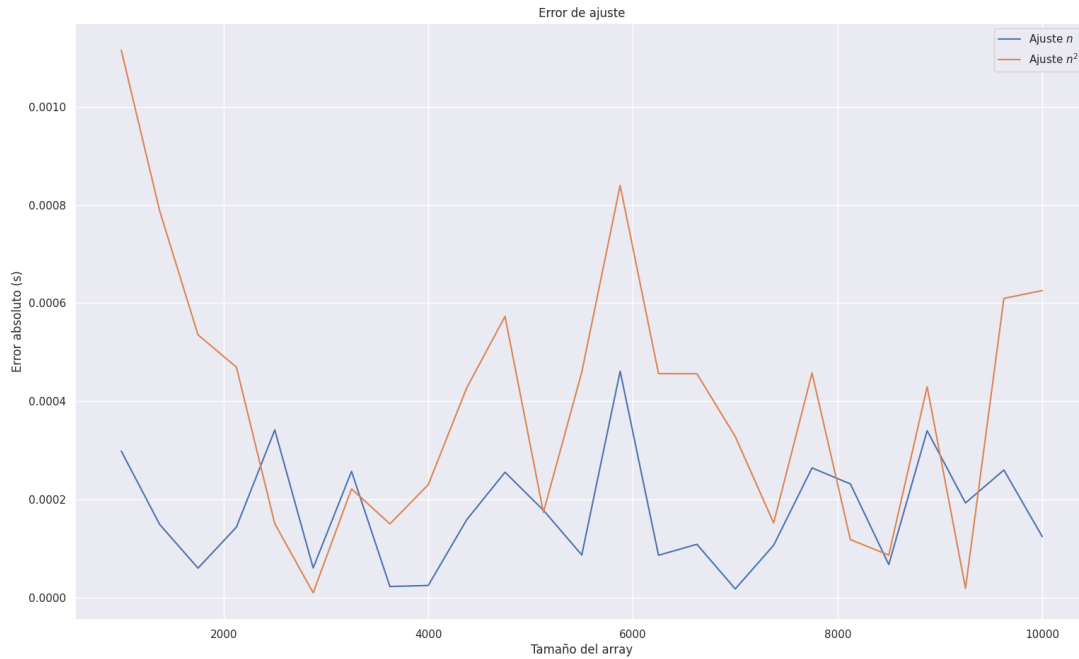


Figura 7: Error cuadrático para ambas funciones.

5.4.2. Caso cuadrático

Por otro lado, se realizaron mediciones para verificar empíricamente la complejidad del peor caso, $O(n^2)$. Se generaron conjuntos de datos tales que $\forall x_i, f(j) < x_i \forall j < i$. Esto obliga a recorrer todos los valores entre 1 e i en cada iteración, produciendo el escenario más desfavorable posible.

Como se observa en las figuras 8 y 9, en este caso la función $T(n) = n^2$ es la que mejor se ajusta. El error cuadrático total obtenido para esta última fue $r \approx 0,47173$, mientras que el ajuste con $T(n) = n$ arrojó un error de $r \approx 4,41458$.

El error obtenido para cada valor de entrada, en ambas funciones, se muestra en la figura 10.

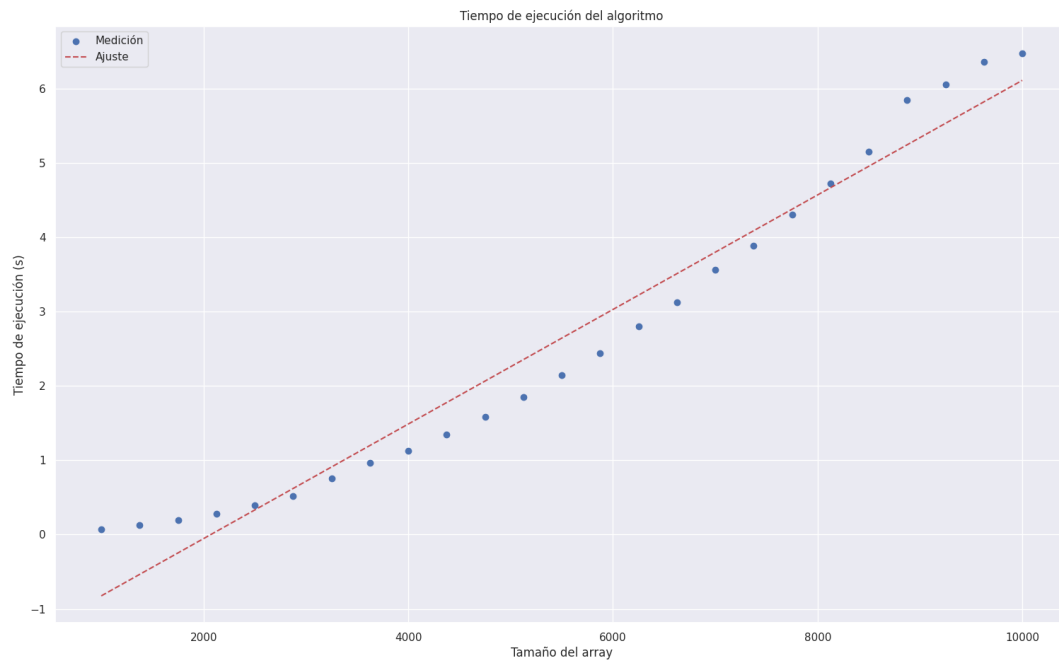


Figura 8: Ajuste del tiempo de ejecución para $T(n) = n$.

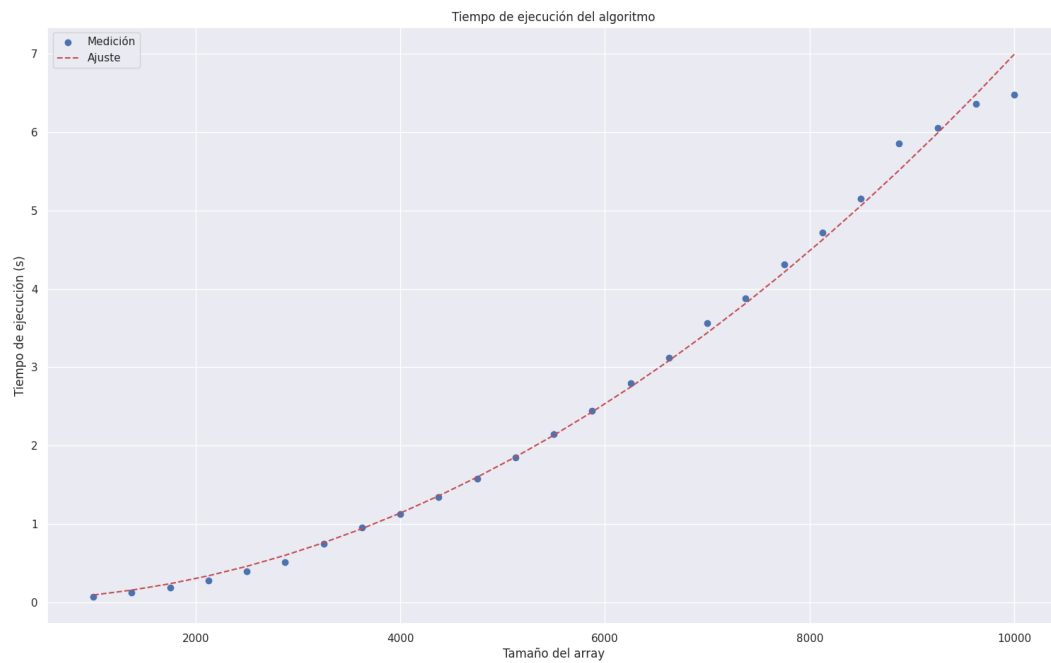


Figura 9: Ajuste del tiempo de ejecución para $T(n) = n^2$.

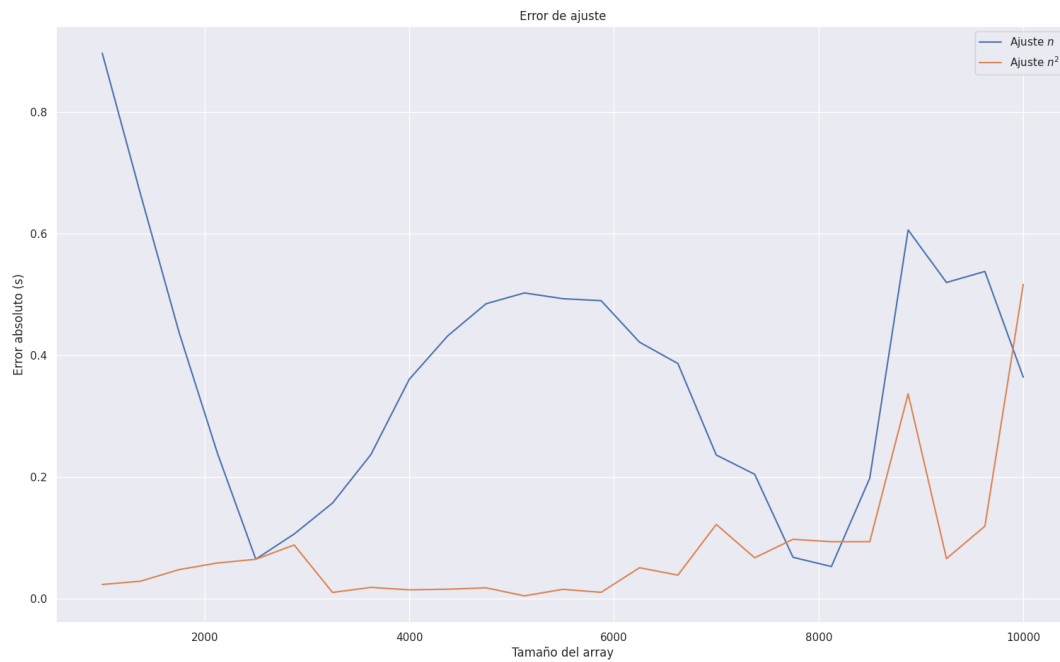


Figura 10: Error cuadrático para ambas funciones.

Para evitar sobrecargar con figuras esta sección, se incluirá un anexo con casos adicionales, detallando las funciones $f(\cdot)$ y $g(\cdot)$ utilizadas en cada medición, junto con sus respectivos errores cuadráticos.

6. Anexo

6.1. Caso Lineal

6.1.1. $g(\cdot)$ lineal y $f(\cdot)$ es $n \times \log(n)$

- Error cuadrático total para n : 0.00018455297166875573
- Error cuadrático total para n^2 : 0.0021599660416998967

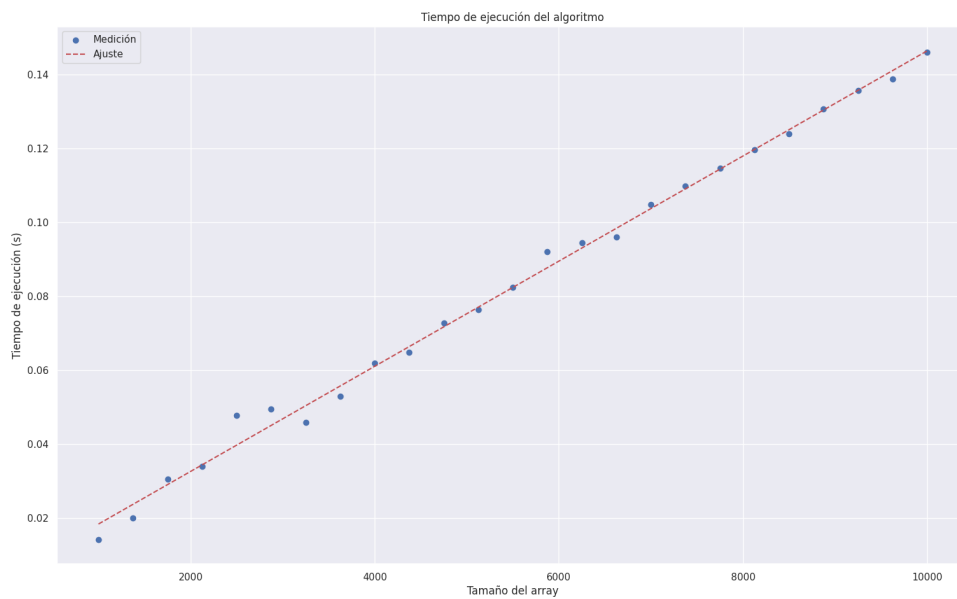


Figura 11: Ajuste con $T(n) = n$ para $g(\cdot)$ lineal y $f(\cdot)$, $n \times \log(n)$

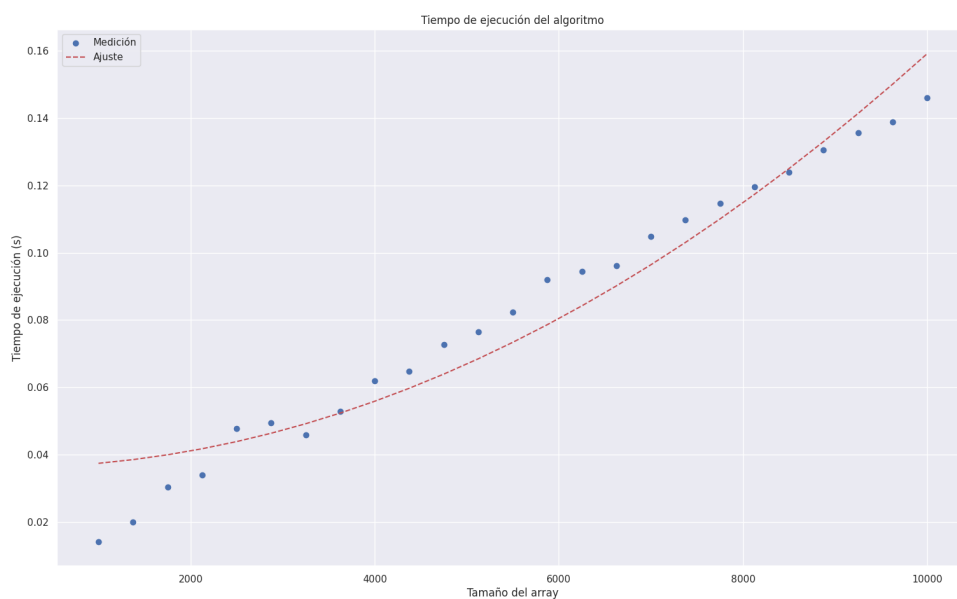


Figura 12: Ajuste con $T(n) = n^2$ para $g(\cdot)$ lineal y $f(\cdot)$, $n \times \log(n)$

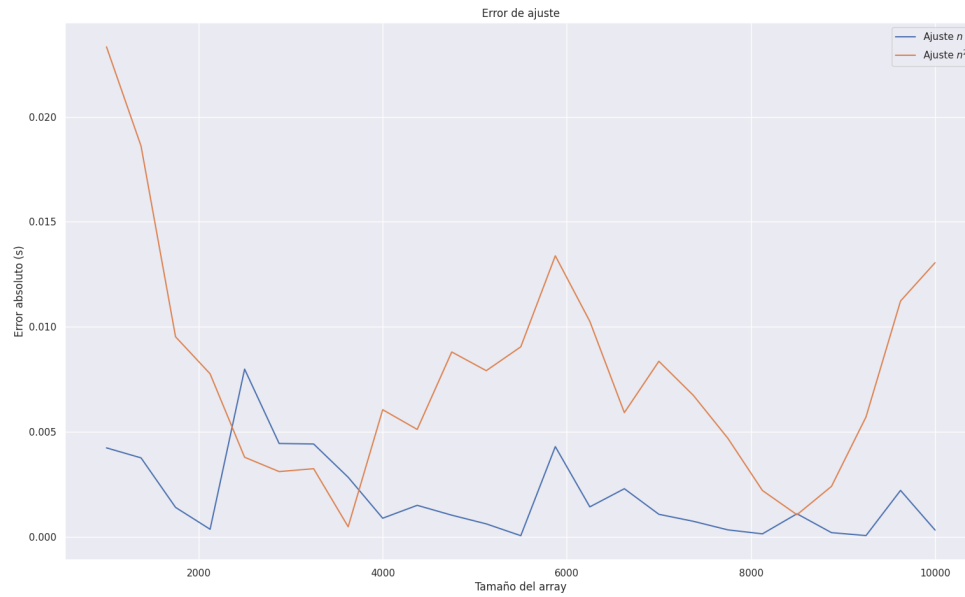


Figura 13: Error de ajuste para $g(\cdot)$ lineal y $f(\cdot)$, $n \times \log(n)$

6.1.2. $g(\cdot)$ cuadrática y $f(\cdot)$ es cúbica

- Error cuadrático total para n : 0.00025591828746134883
- Error cuadrático total para n^2 : 0.0007991220307682418

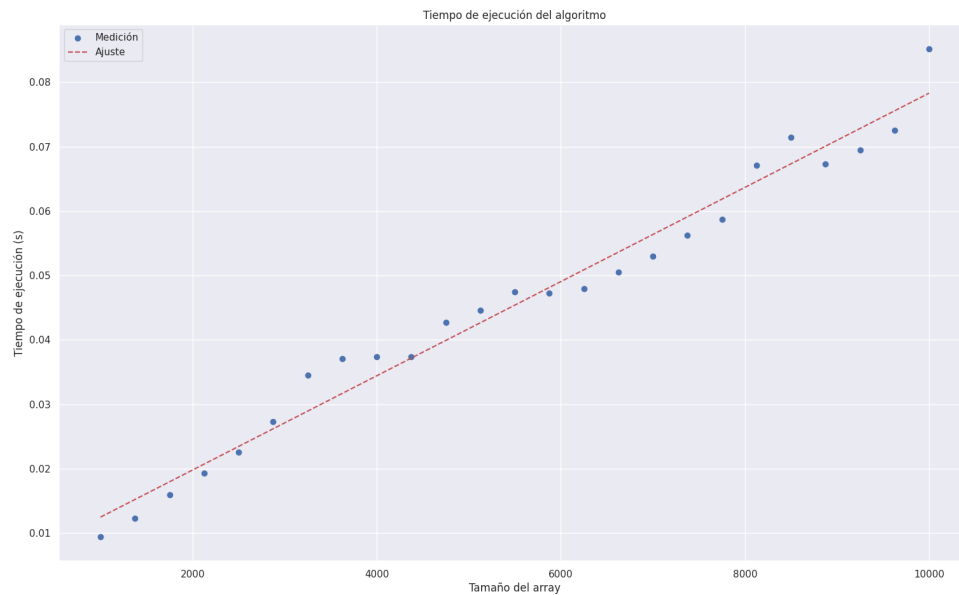


Figura 14: Ajuste con $T(n) = n$ para $g(\cdot)$ cuadrática y $f(\cdot)$ cúbica

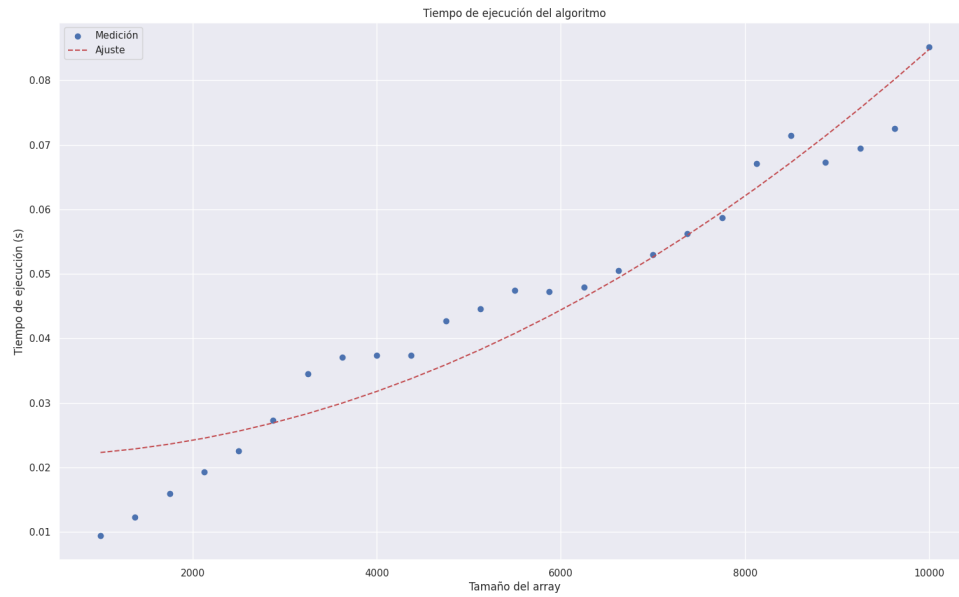


Figura 15: Ajuste con $T(n) = n^2$ para $g(\cdot)$ cuadrática y $f(\cdot)$ cúbica

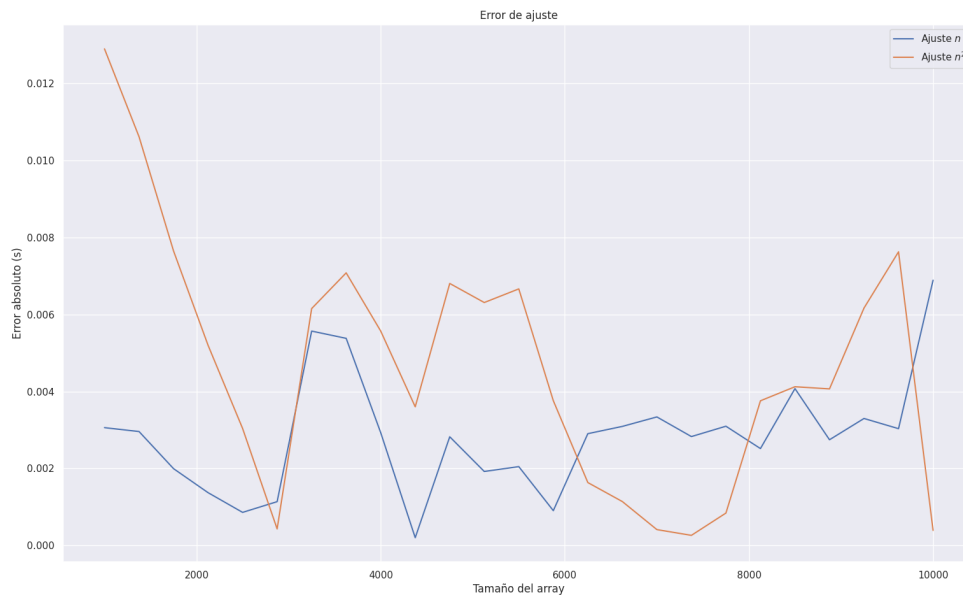


Figura 16: Error de ajuste para $g(\cdot)$ cuadrática y $f(\cdot)$ cúbica

6.2. Caso Cuadrático

6.2.1. $g(\cdot)$ cuadrática y $f(\cdot)$ es lineal

- Error cuadrático total para n : 4.446982771726068
- Error cuadrático total para n^2 : 0.7620720039169853

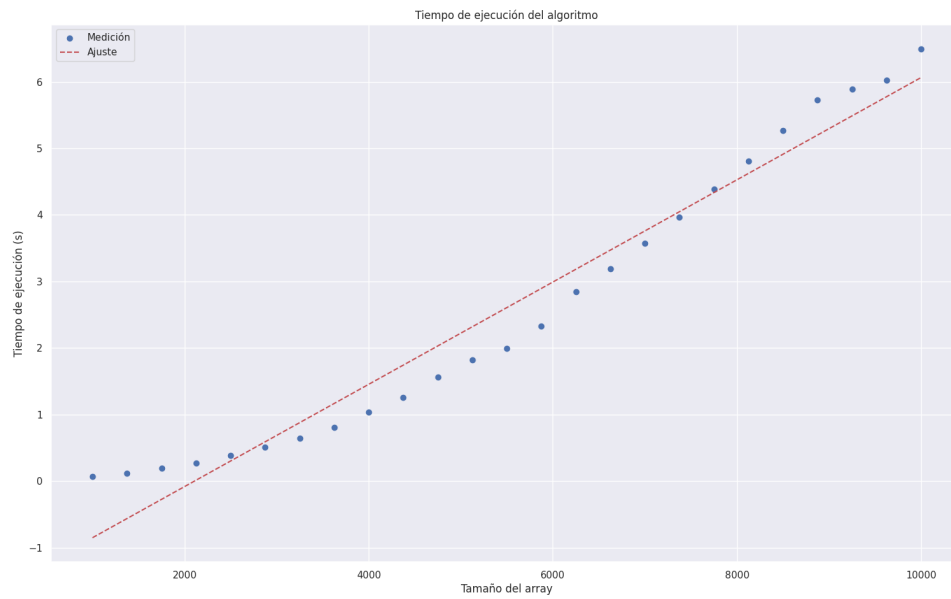


Figura 17: Ajuste con $T(n) = n$ para $g(\cdot)$ cuadrática y $f(\cdot)$ lineal

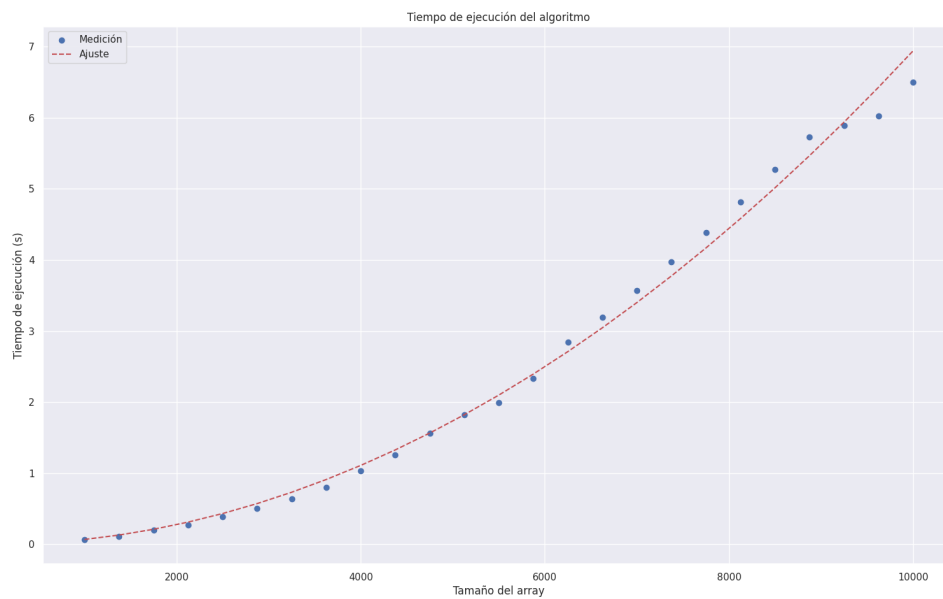


Figura 18: Ajuste con $T(n) = n^2$ para $g(\cdot)$ cuadrática y $f(\cdot)$ lineal

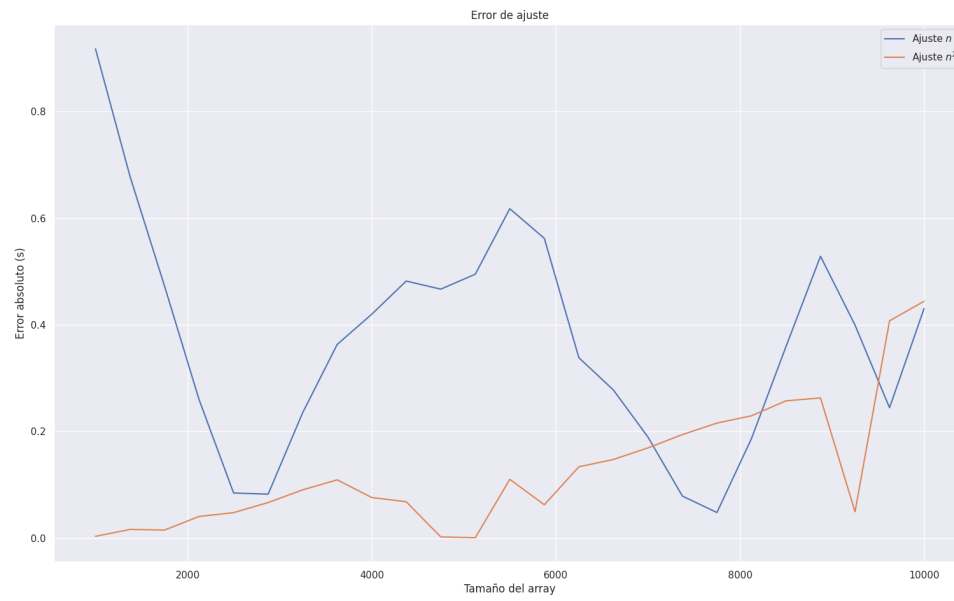


Figura 19: Error de ajuste para $g(\cdot)$ cuadrática y $f(\cdot)$ lineal